

Card Based Dungeon-Crawling Game

Gustavo Antunes, nº50485

André Brito, nº50487

Orientadores: Pedro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Card Based Dungeon-Crawling Game

50485 Gustavo Antunes

50485 André Brito

Orientador: Pedro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

Agradecimentos

Em primeiro lugar, obrigado a todos os que acreditaram que seria possível executar este projeto, que acreditaram na ideia, e que nos ajudaram ao longo deste percurso, em particular, o orientador Pedro Pereira. Não tendo sido uma jornada sem os seus altos e baixos, o que mais se destacou não foram as dificuldades e problemas, mas o apoio que recebemos ao longo da execução e conclusão deste projeto.

Às outras pessoas que também participaram indiretamente desta jornada, e que a tornaram mais fácil por quaisquer razões, obrigado. Se conseguimos acabar o que conseguimos (a tempo da entrega...), é graças a vocês.

Abstract

This report presents the design and implementation of an application that enables the execution of a game conceived and developed as part of the Project and Seminar course. The challenge was to develop an application that allows different types of client to authenticate, create or search for game tables, and enjoy a real-time multiplayer game with a unique game-play experience that combines several concepts not commonly found together.

A aplicação aborda tópicos essenciais do desenvolvimento de software, como a imutabilidade dos dados, código concorrente, a partilha de código comum a vários componentes de uma aplicação, e a fácil expansibilidade do código escrito totalmente em Kotlin.

Resumo

Este relatório apresenta o desenho e implementação de uma aplicação que permite a execução de um jogo planeado e desenvolvido no âmbito da unidade curricular Projeto e Seminário. O desafio era realizar uma aplicação que permitisse que tipos de cliente diferentes fossem capazes de se autenticar, criar, ou procurar mesas de jogo, e desfrutar de um jogo multijogador em tempo real com uma vertente única, que mistura vários conceitos diferentes e que não costumam coexistir.

A aplicação desenvolvida tem como base tópicos essenciais do desenvolvimento de software, como a imutabilidade dos dados, código concorrente, programação funcional e reativa, e a fácil expansibilidade do código escrito totalmente em Kotlin.

Utilização de Inteligência Artificial

A performance das ferramentas AI tem sido, na sua grande maioria, decepcionante, mesmo restringindo o uso destas ferramentas a tarefas simples, por exemplo, a realização de testes automáticos para elementos modulares, ou para a criação de diagramas PlantUML e outros elementos de documentação. É importante notar que quaisquer crítica é feita à opção gratuita de cada modelo.

É importante ressaltar em primeiro lugar que há uma diferença significativa no uso das ferramentas AI no browser, das que estão instaladas localmente na máquina ou na IDE. O tempo e recursos que são gastos a contextualizar as ferramentas no browser não compensa, de todo, a sua utilização - ao invés de se gastar tempo a escrever código, usa-se a mesma quantidade de tempo para retificar, e muitas vezes completar, o código gerado.

Fora do contexto de utilização previamente descrito, nenhum modelo, com ou sem contexto, foi capaz de cumprir com todas as exigências que nos tínhamos proposto a seguir, por exemplo, ignorando a tentativa de escrever código funcional, ou adicionando mutabilidade onde não era necessário.

A utilização correta destas ferramentas é crucial para acelerar o processo de trabalho, principalmente nos dias de hoje em que tudo se está a ajustar lentamente a AI, mas também é importante notar que confiar demasiado nestas ferramentas pode contribuir para uma deteriorização do sentido critico, e para uma maior dificuldade em pensar em possíveis soluções para um problema de forma independente, o que consideramos ser qualidades essenciais para um engenheiro.

Índice

1	Introdução	1
1.1	O Jogo	1
1.2	Como Jogar	4
1.3	Organização do Documento	5
2	Enquadramento	7
2.1	Conceitos Importantes	7
2.2	Objetivo do projeto	7
3	Arquitetura	9
3.1	Tecnologias usadas	9
3.2	Cliente	11
3.3	Servidor	11
3.4	Base de dados	12
3.5	Requisitos Funcionais	12
4	Implementação	13
4.1	Organização do projeto	13
4.1.1	Compose App	14
4.1.2	Server	14
4.1.3	Shared	14
4.2	FrontEnd/Cliente	15
4.2.1	Navegação	15
4.2.2	ClientAPI	17
4.2.3	ViewModel	18
4.2.4	Ecrãs	21
4.3	BackEnd/Servidor	22
4.3.1	Base de dados	22
4.3.2	Autenticação	27
4.3.3	Comunicação cliente-servidor	28
4.3.4	Controle de Erros	30

4.3.5	Testes	30
4.4	BackEnd/Shared	31
4.4.1	Domínio	31
4.4.2	Expansibilidade	35
5	Conclusões	39
5.1	Dificuldades no projeto	39
5.2	Objetivos não implementados	40
5.2.1	Lógica de Jogo	40
5.2.2	Aspetos técnicos	41
5.3	Melhorias a fazer	41
5.4	O que aprendemos	43
	Referências	45

Lista de Figuras

1.1	Peças para os clientes construírem o tabuleiro	1
1.2	Um estado possível para o tabuleiro de jogo	1
1.3	Representação conceptual de um Personagem no jogo	2
1.4	Representação conceptual de um Item no jogo	3
1.5	Exemplo de efeito ativável pelos personagens no tabuleiro	3
1.6	Comparação entre um personagem Básico, e um personagem Raro	3
4.1	Organização do Projeto	13
4.2	Transição de estados do Cliente	16
4.3	Transição de estados do Cliente	16
4.4	Transição de estado da Interface do Jogo para colocar uma carta no tabuleiro	18
4.5	Transição de estado da Interface do Jogo para inspecionar elementos na mão do jogador ou no tabuleiro	19
4.6	Transição de estado da Interface do Jogo para realizar o movimento de um Personagem	19
4.7	Transição de estado da Interface do Jogo durante a realização de uma Batalha	20
4.8	Transição de estado da Interface do Jogo para apresentar a tela de Fim de Jogo	20
4.9	Diagrama EAbd	26
4.10	Diagrama PUMML das classes Table e User	31
4.11	Diagrama PUMML da classe Board	32
4.12	Diagrama PUMML da interface Character	32
4.13	Diagrama PUMML da interface Card	33
4.14	Diagrama PUMML da classe Battle	34
4.15	Diagrama PUMML da classe Game	35

Lista de Tabelas

4.1	Tabela Users	23
4.2	Tabela AuthUsers	23
4.3	Tabela Tables	23
4.4	Tabela Participants	23
4.5	Tabela Games	24
4.6	Tabela GamePlayers	24
4.7	Tabela GameSpectators	24
4.8	Tabela BoardTiles	24
4.9	Tabela BoardCharacterItems	25
4.10	Tabela BoardCharacterModifiers	25

Capítulo 1

Introdução

1.1 O Jogo

No *Card Based Dungeon Crawling Game*, um jogador terá um conjunto de personagens à sua disposição, e terá de construir e percorrer o tabuleiro de jogo juntamente com os restantes jogadores. Nas Figuras abaixo estão representados exemplos das peças (*Fig. 1.1*) que os jogadores terão para construir o tabuleiro (*Fig 1.2*), e toda a informação referente a um personagem (*Fig 1.3*) controlado por eles.

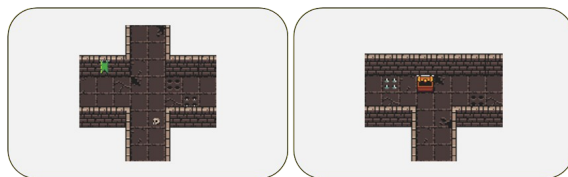


Figura 1.1: Peças para os clientes construírem o tabuleiro



Figura 1.2: Um estado possível para o tabuleiro de jogo



Figura 1.3: Representação conceptual de um Personagem no jogo

Cada personagem tem um conjunto de atributos e habilidades diferentes, o que exige uma mentalidade e estilo de jogo diferente por parte do jogador. Os quatro atributos principais de um personagem são:

- **Pontos de Vida (HP):** como o nome indica, são os pontos de vida de um personagem. Quando este valor chega a 0 durante a exploração da área de jogo, ele é considerado incapaz de continuar e é removido. Quando este valor chega a 0 durante uma batalha, o personagem é derrotado e perde um ponto de vida permanentemente;
- **Ataque (DMG):** dano que causa a um oponente num ataque durante uma batalha;
- **Defesa (DEF):** quantidade de pontos de vida protegidos de um ataque;
- **Velocidade (SPE):** determina a ordem dos turnos do jogo, com os personagens mais rápidos a agir primeiro, ou determina a ordem de ataque dos personagens numa batalha. Além disso, influencia na probabilidade de um personagem fugir de uma batalha, ou passar despercebido durante a exploração;

Os personagens também são capazes de utilizar itens para se fortalecerem, ou podem ativar efeitos especiais durante a exploração do tabuleiro que aumenta os seus atributos, ou diminuir os atributos dos oponentes. Alternativamente, estabelecidas as condições certas, personagens têm a capacidade de aumentar os valores dos seus atributos ao evoluir, alterando a sua habilidade passiva e ganhando a capacidade de equipar itens mais poderosos. Nas Figuras 1.4 a 1.6 que se seguem, está a representação de toda a informação de um Item no jogo (Fig. 1.4), um exemplo de efeito especial que os jogadores podem ativar no tabuleiro (Fig. 1.5), e a comparação entre um personagem antes e depois de "evoluir" (Fig. 1.6).

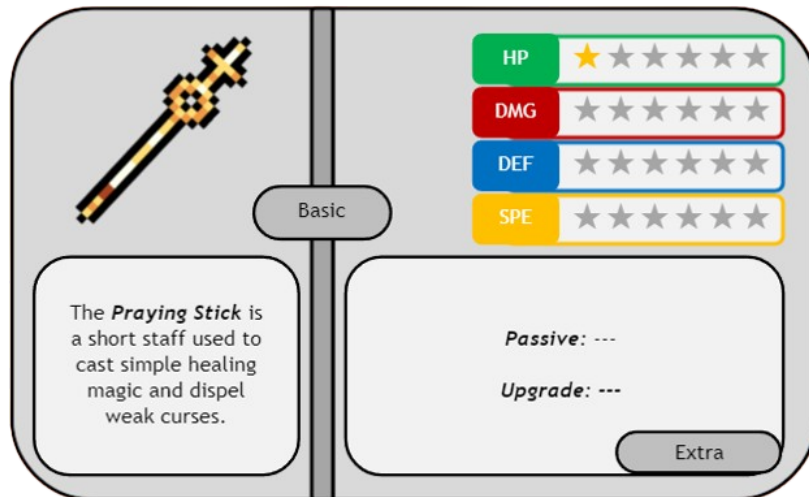


Figura 1.4: Representação conceptual de um Item no jogo

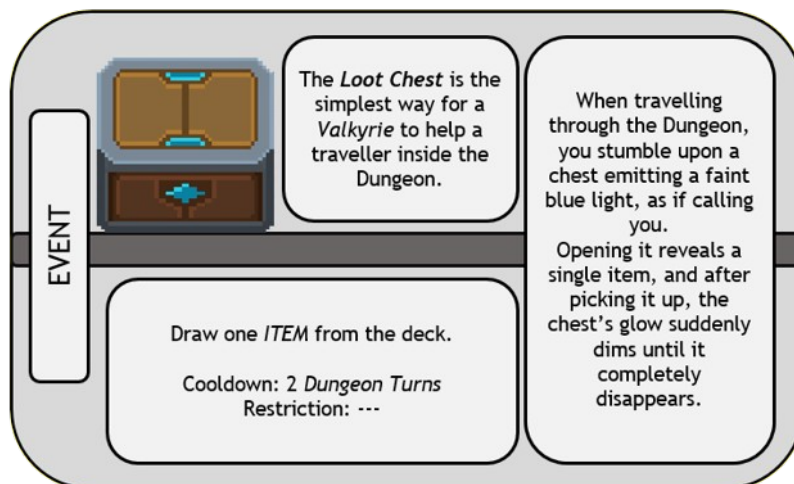


Figura 1.5: Exemplo de efeito ativável pelos personagens no tabuleiro

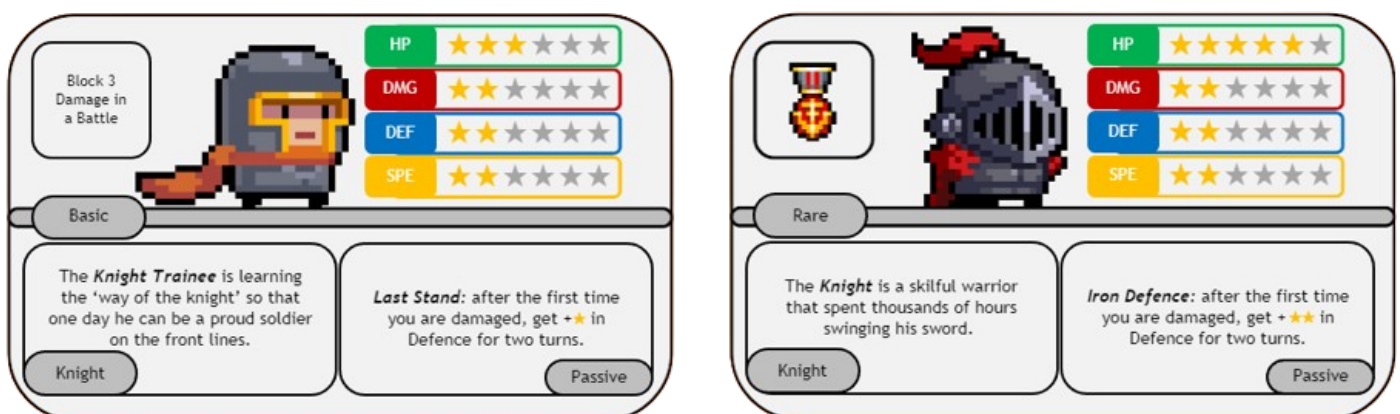


Figura 1.6: Comparação entre um personagem Básico, e um personagem Raro

O objetivo do jogo é simples: derrotar os personagens de todos os outros jogadores, ou obter todos os anéis e voltar ao início da masmorra. Assim sendo, apenas um jogador pode vencer este jogo.

1.2 Como Jogar

Como a maior parte dos jogos de tabuleiro, CBDCG é jogado por turnos, e cada jogador terá um conjunto de ações que pode realizar durante o seu:

- **Fase de Construção:** o jogador retira uma carta do baralho das peças do tabuleiro e coloca-a na sua mão. Além de não ser sempre obrigado a colocar uma peça no tabuleiro durante o seu turno, o jogador pode colocar qualquer número de peças de uma só vez, desde que todas liguem ao resto do tabuleiro;
- **Fase de Substituição:** o jogador decide se quer continuar com o seu personagem, ou se prefere trocar para outro personagem na sua mão. O jogador também pode equipar ou desequipar itens de qualquer um dos seus personagens;
- **Fase de Movimento:** o jogador pode mover o seu personagem. Antes de passar por uma peça com um efeito especial, este é automaticamente ativado. Personagens podem desafiar outros personagens adjacentes. Qualquer personagem adjacente a um dos combatentes também se pode juntar à batalha;

Durante uma batalha, um jogador pode executar uma de três ações com o seu personagem:

- **Atacar:** tenta atacar um oponente. O valor de DMG do atacante é reduzido primeiro ao valor de DEF do defensor, e só depois ao de HP;
- **Aguardar:** assume uma postura defensiva, aumentando a sua DEF em 1 ponto, ou recuperando a sua DEF para o valor original. Esta ação não pode ser usada consecutivamente;
- **Fugir:** tentativa de abandonar o combate. A probabilidade de sucesso vai aumentar ou diminuir dependendo da diferença entre o valor de SPE do personagem e o valor de SPE do oponente com maior velocidade;

1.3 Organização do Documento

O restante relatório encontra-se organizado da seguinte forma:

- Cap 2 Enquadramento: Conceitos de contextualização e objetivos do projeto.
- Cap 3 Arquitetura: Explicação da arquitetura do projeto e das tecnologias utilizadas.
- Cap 4 Implementação: Explicação de todos os detalhes de implementação relevantes, assim como justificação das nossas decisões de implementação.
- Cap 5 Conclusões: Conclusões finais do relatório, pontos sobre o projeto no geral e reflexão acerca dos objetivos não cumpridos, de possíveis melhorias, e das dificuldades durante o desenvolvimento.

Capítulo 2

Enquadramento

2.1 Conceitos Importantes

O CBDCG é inspirado em vários conceitos já muito explorados no que diz respeito a jogos, tanto a nível físico como digital:

- **Jogo baseado em Cartas:** num jogo baseado em cartas, a maioria dos elementos são representados através de cartas. Jogos familiares como *UNO!* [1] ou *Trading Card Games* como *Magic: The Gathering* [2] são excelentes exemplos deste tipo de género;
- **Dungeon Crawler:** um *dungeon crawler* é um género de jogo baseado na exploração de uma área repleta de inimigos hostis ao jogador, tesouros, e objetivos que devem ser cumpridos. Diferente dos jogos baseados em cartas, este género é mais comum em formato digital, com representantes como *The Binding of Isaac* [3] por exemplo, do que em formato físico;
- **Jogos por Turnos:** num jogo por turnos, ao invés de todos os jogadores jogarem ao mesmo tempo, cada um tem um momento alocado para agir. Jogos como *Xadrez* [4], na vertente de jogos de tabuleiro, ou o combate em *Baldur's Gate* [5], no que diz respeito a Dungeon-Crawlers, são excelentes exemplos deste género;

2.2 Objetivo do projeto

O objetivo do nosso projeto, é provar que com o conhecimento obtido durante o nosso curso no ISEL, é suficiente para desenvolver um jogo multi-jogador multiplataforma, assim como suficiente para investigar e aprender quaisquer tecnologias novas que achemos adequadas para qualquer projeto que realizemos no futuro. Pretendemos provar, através da nossa implementação, que conseguimos tomar decisões que fazem sentido para o projeto tanto em termos de tecnologia como em termos de arquitetura. Mesmo sem completar todas as nossas ideias

iniciais, focámos-nos em construir uma aplicação fácil de expandir, e cuja estrutura facilita adicionar ideias.

Capítulo 3

Arquitetura

3.1 Tecnologias usadas

Kotlin Multiplatform

Kotlin Multiplatform [6], ou *KMP*, é uma *framework* desenvolvida pela JetBrains [7] que permite desenvolver código comum a várias plataformas utilizando apenas Kotlin.

Um projeto multiplatforma é dividido em três módulos principais: *composeApp* (o Cliente), *server* (o Servidor) e *shared*.

O módulo *shared* possui código que será compartilhado entre cliente e servidor, essencial para evitar escrever código duplicado e para permitir uma comunicação simplificada entre cliente e servidor.

O módulo *server* disponibiliza uma *API* que será consumida pelas aplicações cliente, processa os pedidos destes clientes, executa a lógica de negócio, e gere todo o sistema de acesso à base de dados.

O módulo *composeApp* possui uma implementação do cliente para cada plataforma (JVM, Android, iOS, JS, ...), e código que é compartilhado entre todos os clientes.

Esta organização facilita a criação e manutenção de testes unitários para o cliente ou servidor, dado que estão em módulos independentes. Isto significa que os elementos partilhados por cliente e servidor só precisam de ser testados uma única vez, o cliente não necessita de realizar pedidos ao servidor para ser testado, tendo por exemplo uma implementação temporária da camada de comunicação, e o servidor não tem a necessidade de ter um cliente em execução para lhe fazer pedidos, uma vez que os contratos e os modelos de dados são partilhados.

Ktor

Uma vez que estamos a usar *Kotlin Multiplatform*, faz sentido utilizar a biblioteca *Ktor* [8] para criar aplicações Cliente e Servidor, uma vez que também foi um produto produzido pela *JetBrains* e desenvolvido para ser usado em projetos com arquitetura *KMP*.

Esta biblioteca permite a adição de funcionalidades, como por exemplo, *routing*, serialização de dados, ou tratamento de exceções, de forma modular através da instalação de *plugins*, ou seja, permite criar um cliente ou servidor extremamente configurável.

A principal vantagem de utilizar *ktor*, e não *http4k* [9], que foi uma das outras alternativas exploradas para este projeto, é que o primeiro utiliza *coroutines* do *Kotlin* como mecanismo de concorrência, o que significa que permite a execução de operações de forma assíncrona, algo que não é disponibilizado diretamente pela outra opção.

Exposed

Para realizar a implementação e comunicação com a base de dados, escolhemos a biblioteca *Exposed* [10].

As várias vantagens desta biblioteca aplicam-se a qualquer projeto *Kotlin* que interaja com bases de dados *SQL*, como é o caso do nosso. A principal razão da existência da biblioteca é que permite desenvolver código de acesso à base de dados em *Kotlin*, ou seja, o código será compilado ao mesmo tempo que o resto do projeto, evitando potenciais erros de runtime. É a *Domain Specific Language* do *Exposed* que torna as *queries Type-Safe*, obrigando que todos os dados estejam corretamente preenchidos antes do programa ser colocado em execução.

As outras vantagens estão no mapeamento objeto-relação utilizado para interagir com a base de dados, ou seja, os objetos da base de dados encontram-se mapeados na forma de objetos do domínio, simplificando o desenvolvimento. A *framework* também permite a utilização de *Data Access Objects* para tornar o acesso e manipulação de dados em objetos do domínio, tornando o processo mais intuitivo e fácil de acompanhar.

A alternativa considerada inicialmente, por estarmos mais familiares com a sua utilização, era a biblioteca *jdbi* [11], uma *framework* para *Java* construída sobre a *API* padrão também do *Java*, *JDBC*. Apesar de ambas serem alternativas viáveis, já que bibliotecas *Java* podem ser usadas em código *Kotlin*, *jdbi* é ligeiramente superior a *JDBC*, dado que simplifica o mapeamento de resultados em objetos de domínio, a execução de *queries SQL*, e a gestão de transações. Ainda assim, nenhuma delas permite que as *queries* (escritas em *psql*) sejam verificadas em tempo de compilação, e portanto qualquer erro só será detetado posteriormente quando esta for invocada em *runtime*.

Jetpack Compose

Dado que a *framework* utilizada foi *Kotlin Multiplatform*, também temos a oportunidade de utilizar *Compose Multiplatform* para a partilha da interface gráfica entre vários clientes distintos. Assim, basta realizar apenas uma iteração da interface em *Kotlin*, que a *framework* permite a execução nativa em diferentes plataformas, criando uma experiência semelhante para qualquer tipo de cliente.

O *Jetpack Compose* [12] tem a vantagem de ser um modelo reativo e modular, o que significa que uma alteração no estado da aplicação pode provocar recomposições no ecrã de determinados componentes, e noutros não. A modularidade também é importante porque permite a reutilização de componentes do ecrã.

Reflect

No nosso projeto utilizamos a biblioteca *reflect*, para construir uma estrutura polimorfica, que reflete sempre todos os membros dessa estrutura que se encontram implementados.

Esta biblioteca foi utilizada por garantir que adicionar membros a esta estrutura, apenas é necessário construir dito membro, que é automaticamente detetado e mapeado como todos os outros, expondo a função ao resto da aplicação. Utilizar reflexão foi extremamente útil, pois permite implementar a estrutura polimorfica desejada, com o mínimo de esforço necessário para adicionar membros á mesma. Não utilizando, reflexão seria necessário adicionar muito mais lógica por cada membro adicionado.

3.2 Cliente

Cada plataforma cliente é um ponto de entrada à nossa aplicação, cada uma responsável por adequar e integrar o código desenvolvido para o respetivo dispositivo e sistema operativo. Por ser um projeto *KMP*, e como foi devidamente referido, uma grande parte do código é compartilhado por todos os clientes, normalmente, código relacionado com a interface gráfica, a interação com o utilizador, e a comunicação com o server através da *API* disponibilizada pelo mesmo.

3.3 Servidor

O servidor, como foi dito anteriormente, possui várias funções no contexto de um projeto *KMP*, nomeadamente, a definição das rotas da *API*, toda a lógica de verificação das regras de negócio, tratando de lançar, captar, e tratar exceções para construir uma resposta adequada para os clientes, e finalmente, gerir o código de acesso à base de dados.

3.4 Base de dados

Para armazenar, gerir, e organizar informação de forma persistente, recorreremos à utilização de uma Base de Dados *PostgreSQL* [10]. Utilizando a biblioteca *Exposed* anteriormente mencionada, somos capazes de consultar, atualizar, ou remover informação da base de dados escrevendo exclusivamente código *Kotlin*.

3.5 Requisitos Funcionais

Gestão de Utilizadores

- Criação de conta;
- Autenticação (Login/Logout)

Gestão de Mesas de Jogo

- Criação de uma mesa;
- Listagem de todas as mesas;
- Entrada e saída de uma mesa;
- Alteração do papel a desempenhar no jogo (Player/Spectator);

Lógica de Jogo

- Colocação de cartas na área de jogo (Personagens, Peças, Itens);
- Inspeccionar elementos do jogo, tanto cartas como elementos no tabuleiro;
- Rotação de peças na mão do jogador;
- Movimentação de Personagens na área de jogo;
- Ativar efeitos de uma Peça;
- Trocar de Personagem;
- Desequipar itens de um Personagem;
- Sistema de Combate;
- Habilidades passivas dos Personagens;
- Verificação das condições de vitória

Capítulo 4

Implementação

4.1 Organização do projeto

Explicada no capítulo anterior, a *Figura 4.1* ilustra a arquitetura do projeto *KMP* desenvolvido, assim como a subdivisão de cada módulo e as tecnologias implementadas em cada um.

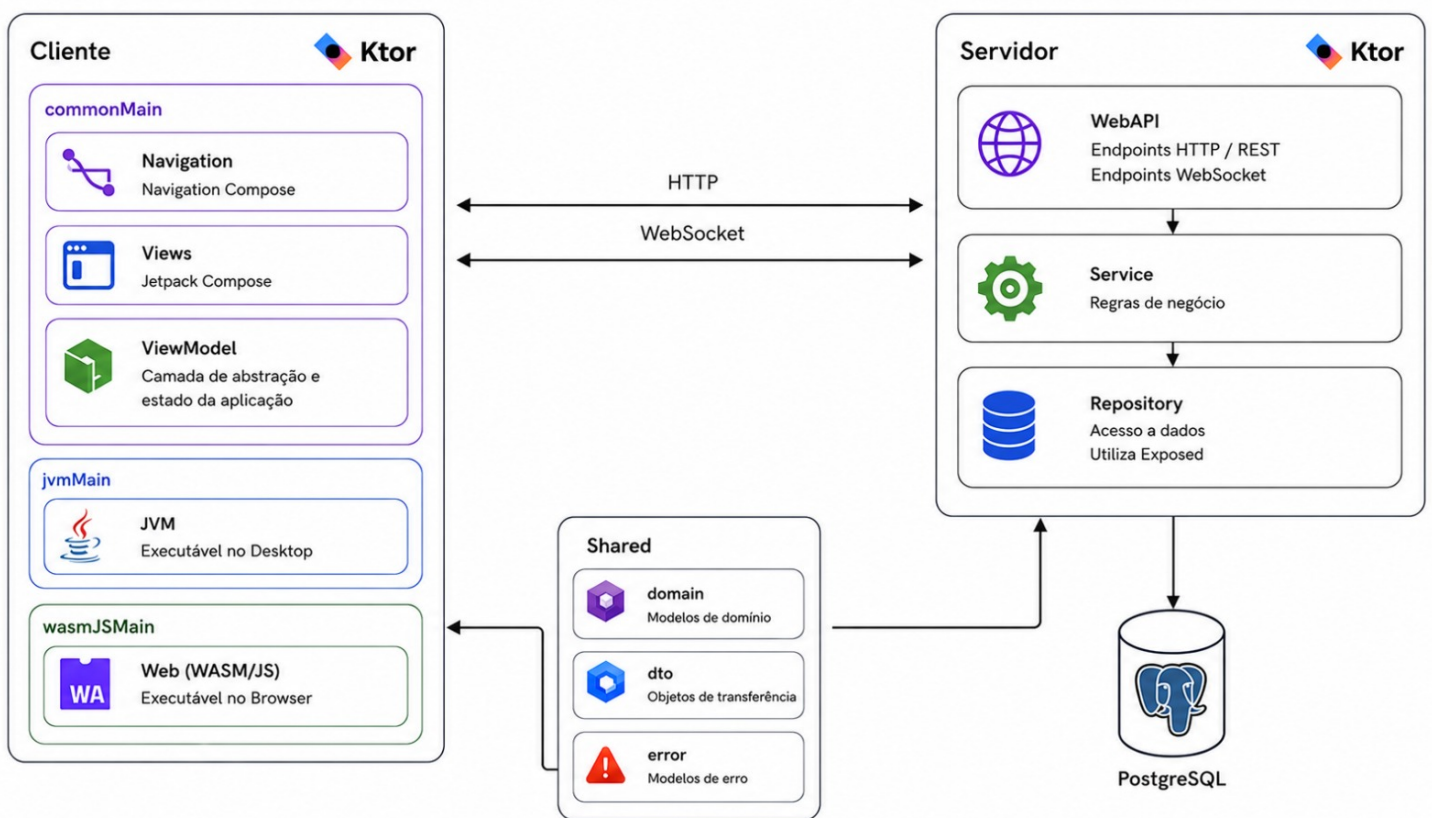


Figura 4.1: Organização do Projeto

4.1.1 Compose App

Este módulo contém a lógica do cliente e os elementos de composição da *User Interface*. Sub divide-se em:

- **commonMain**: componentes comuns a qualquer cliente da aplicação;
- **jvmMain**: cliente *JVM* (Desktop);
- **wasmJsMain**: cliente *wasmJS* (Browser);

O módulo compartilhado subdivide-se ainda em:

- **navigation**: rotas de ecrã e componente de navegação entre ecrãs do cliente;
- **viewmodel**: componente intermédio entre o estado recebido do servidor e a *UI* do cliente;
- **views**: componentes visuais do cliente;

4.1.2 Server

Este modulo contém a arquitetura do servidor e trata de expor os meios de comunicação entre ele e os clientes, implementando a forma como a lógica da aplicação pode ser utilizada. Contém também a camada de acesso a dados. Sub divide-se em:

- **Config**: Ficheiros de configuração;
- **Repository**: Camada de acesso a dados (*DAL*);
- **Service**: Comunicação servidor-Dominio;
- **WebApi**: Comunicação cliente-servidor;
- **Aplication**: Ficheiro que controla e inicia o servidor;

4.1.3 Shared

No modulo shared, encontra se todo o código que é partilhado entre cliente e servidor. Este modulo contém a lógica da aplicação, as definições dos objetos utilizados, e os *Data Transfer Objects* (DTOs) utilizados para a troca de dados com o cliente. Sub divide-se em:

- **Domain**: Lógica da aplicação e defenições de objetos;
- **DTO**: Objetos de comunicação com o cliente;
- **Error**: Definição dos erros lançados durante a aplicação;

4.2 FrontEnd/Cliente

A nossa aplicação tem dois tipos de cliente *Ktor*: *Desktop* (JVM) e *Browser* (wasmJS). Ambos usam uma engine assíncrona e instalam *plugins* para as comunicações *HTTP* e *WebSocket*, e para o armazenamento de recursos (imagens), sendo que o cliente *Desktop* tem a capacidade de guardar estes recursos permanentemente através do *FileStorage*, criando uma pasta local para armazenamento, e o cliente *Browser* está dependente do próprio browser para ter armazenamento permanente.

4.2.1 Navegação

A navegação entre ecrãs foi implementada recorrendo à biblioteca *Navigation Compose* do *Jetpack Compose*. Há três componentes principais no que diz respeito à navegação entre ecrãs:

- **NavHost**: é o contentor que associa o controlador ao conjunto de destinos disponíveis ao cliente;
- **NavController**: o controlador é o componente responsável por controlar o fluxo da navegação, isto é, a navegação entre estados e a manipulação da pilha de navegação;
- **Rotas**: conjunto de destinos que o controlador tem acesso, cada um associado a um elemento visual;

Estes componentes, no entanto, não são suficientes para mudar o estado do cliente. Como foi referido anteriormente, o *Jetpack Compose* é reativo, ou seja, é capaz de detetar e reagir às variações de um *StateFlow*. Um *StateFlow* mantém o valor mais recente de um conjunto de valores emitidos ao longo do tempo de forma assíncrona, ou seja, à medida que são realizadas alterações no servidor, estas alterações são enviadas ao cliente e alteram o *StateFlow*. O cliente está preparado para coletar este valor recém-chegado e verificar em que rota é que o controlador deve estar.

O cliente tem cinco possíveis rotas, ou seja, cinco possíveis ecrãs de destino:

- **Menu**: o ecrã inicial da aplicação, mostrado quando um cliente não está autenticado;
- **Tutorial**: o ecrã destinado à execução de determinado tutorial;
- **Lobby**: o ecrã onde o cliente pode criar ou juntar-se a mesas de jogo;
- **Table**: o ecrã onde o cliente define o seu papel no jogo, e espera que o mesmo comece;
- **Game**: o ecrã apresentado durante a execução de um jogo;

Na *Figura 4.2* abaixo está o diagrama de transição de estado entre estes cinco ecrãs principais, e na *Figura 4.3* o diagrama de transição estado completo entre todos os ecrãs de navegação.

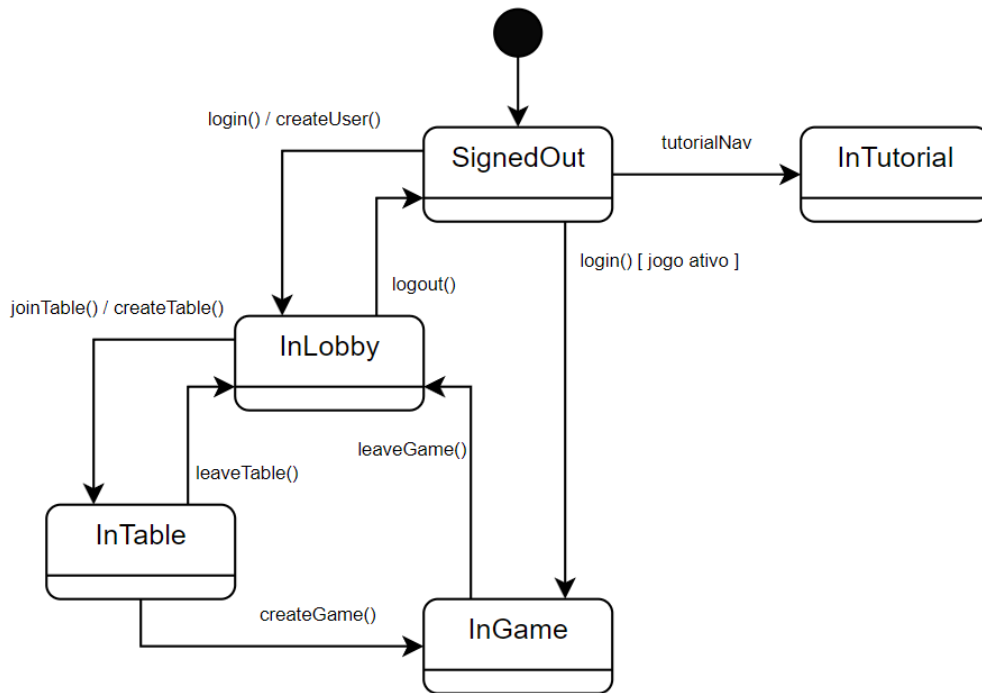


Figura 4.2: Transição de estados do Cliente

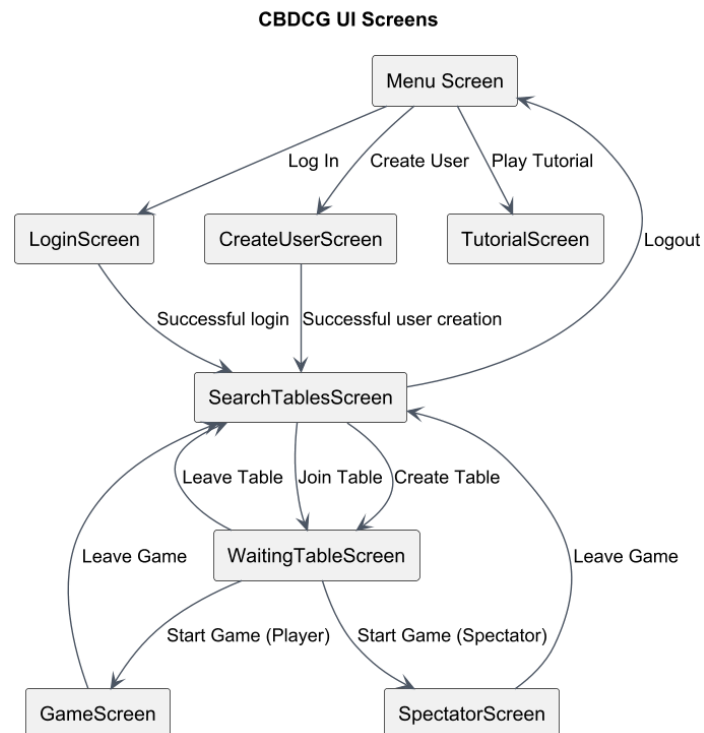


Figura 4.3: Transição de estados do Cliente

Deste modo, há uma divisão de responsabilidades clara, dado que a navegação não depende totalmente da interação de um utilizador com a *UI*, mas sim de uma alteração no estado da aplicação, permitindo atualizar o ecrã em tempo real. Dito isto, ainda há navegação "manual", isto é, dependente de um clique do utilizador, como por exemplo, voltar ao ecrã anterior, ou fechar uma janela de erro, uma vez que estes podem não exigir que seja feito um pedido ao servidor por ser uma operação exclusiva do cliente.

Tanto a organização das rotas como a lógica de navegação entre ecrãs são independentes da plataforma cliente, uma vez que não recorrem a funcionalidades específicas de qualquer sistema operativo ou tipo de dispositivo. Assim sendo, a implementação desta categoria é feita no módulo '*commonMain*' de *composeApp*.

4.2.2 ClientAPI

O ClientAPI é o módulo que abstrai o resto do cliente de qualquer interação com o servidor. É este componente que trata de construir e realizar os pedidos *HTTP* ao servidor, e transforma a resposta numa alteração de estado. De forma semelhante, o ClientAPI também faz a gestão da comunicação WebSocket, e está preparado para receber e decodificar sinais específicos vindos do servidor.

O *ClientAPI* mantém três *StateFlows*:

- **tables**: variável que armazena todas as mesas disponíveis para entrar;
- **currentTable**: variável que armazena a informação da mesa frequentada pelo cliente;
- **game**: variável que armazena a informação do jogo em que o cliente está a participar;

Estes três *StateFlows* sofrem alterações quando o cliente recebe um sinal *WebSocket* vindo do servidor. A comunicação *HTTP*, não sendo assíncrona, não influencia os valores dos *StateFlows*:

- **LobbyTables**: quando uma mesa é criada, eliminada, ou alterada, é necessário atualizar '*tables*';
- **TableInfo**: quando a mesa em que o jogador está é alterada, é preciso atualizar '*currentTable*';
- **TableDeleted**: quando a mesa em que o jogador está é eliminada, é preciso atualizar '*currentTable*';
- **GameInfo**: quando o jogo é atualizado, é preciso atualizar '*game*';

Uma vez que todos os clientes tiram proveito das classes presentes no módulo '*shared*', e acessam as mesmas rotas do 'servidor', então a construção dos pedidos e a interpretação das respostas enviadas pelo servidor serão as mesmas em qualquer cliente, justificando a implementação do *ClientAPI* no módulo '*commonMain*' de *composeApp*.

4.2.3 ViewModel

O *ViewModel* é uma camada intermédia entre a interface gráfica e as restantes componentes da aplicação. A sua principal responsabilidade consiste em gerir o estado da interface de acordo com os resultados obtidos pela comunicação com o servidor. Além disso, ela também disponibiliza operações a ser invocadas pelo cliente nos vários ecrãs da aplicação.

O estado da aplicação contém informação acerca da sessão do cliente, por exemplo, se já está autenticado ou se está numa mesa, e contém informação acerca do estado do ecrã durante um jogo, por exemplo, se o jogador está a inspecionar algum elemento do jogo, ou se está numa batalha. O estado é um *StateFlow*, cujo valor mais recente é o estado atual da interface gráfica.

Nas *Figuras 4.4 a 4.8*, estão representados os estados da interface gráfica do cliente ao longo do jogo, de acordo com as ações efetuadas pelo utilizador.

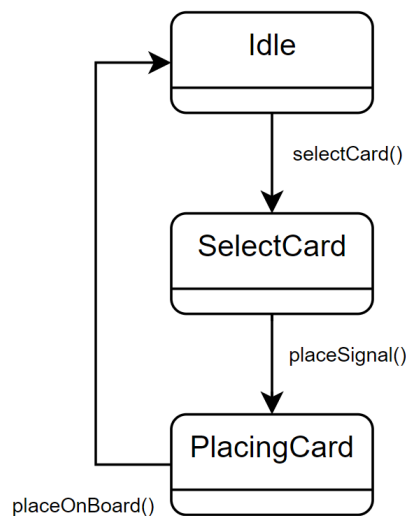


Figura 4.4: Transição de estado da Interface do Jogo para colocar uma carta no tabuleiro

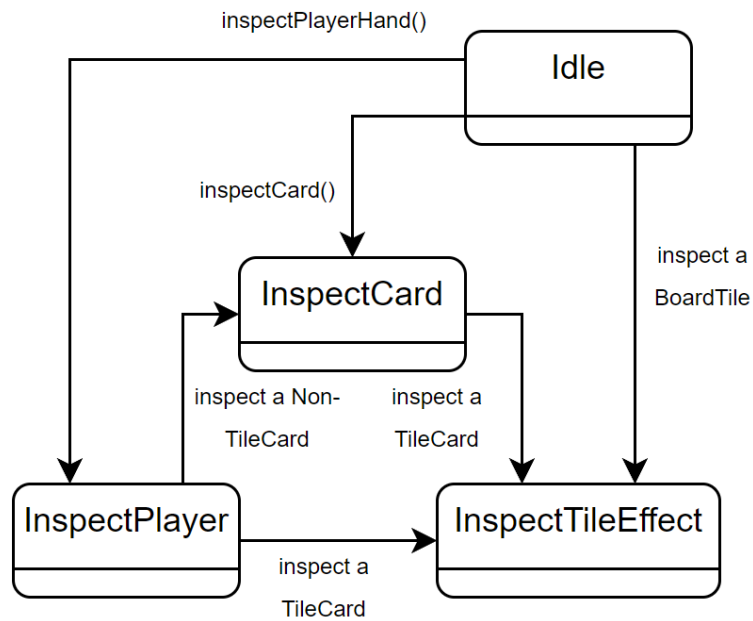


Figura 4.5: Transição de estado da Interface do Jogo para inspecionar elementos na mão do jogador ou no tabuleiro

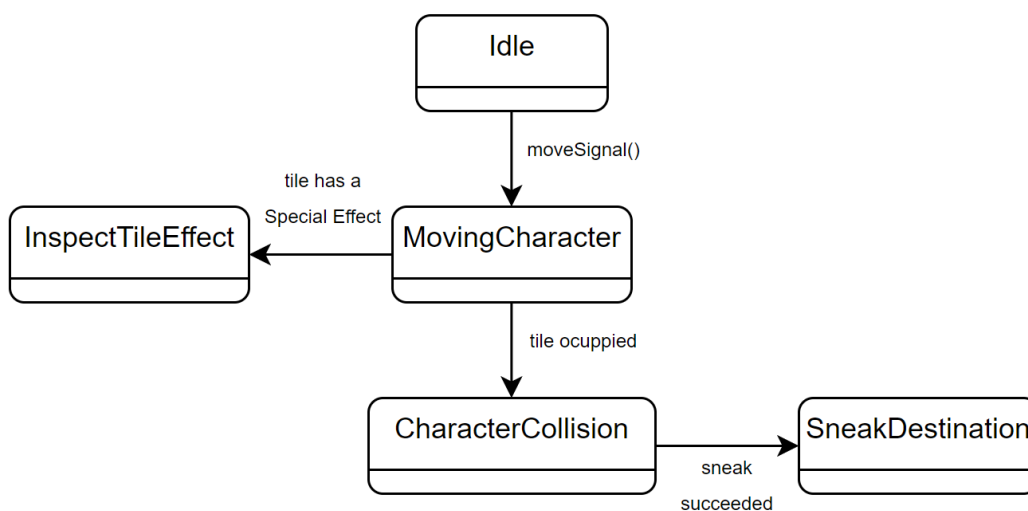


Figura 4.6: Transição de estado da Interface do Jogo para realizar o movimento de um Personagem

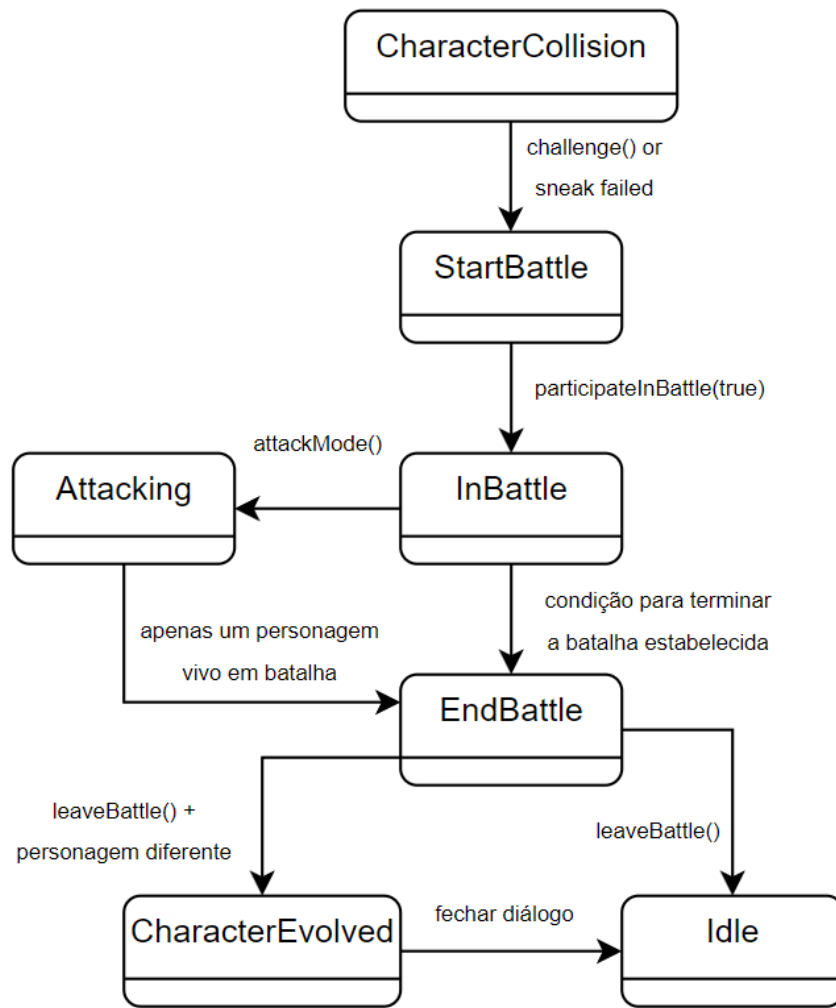


Figura 4.7: Transição de estado da Interface do Jogo durante a realização de uma Batalha

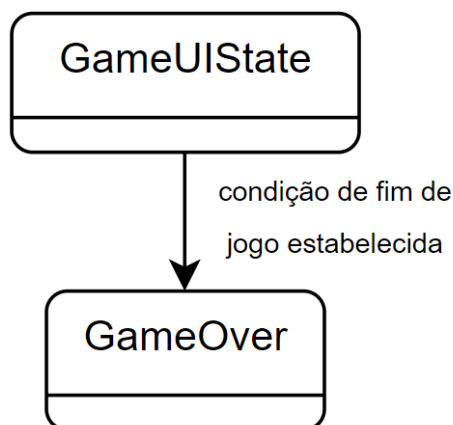


Figura 4.8: Transição de estado da Interface do Jogo para apresentar a tela de Fim de Jogo

O *ViewModel* tem a responsabilidade de interpretar a informação captada no *ClientAPI* e adaptar o estado atual da interface gráfica. Pelo estado se tratar de um *StateFlow*, como referido anteriormente, quando este é alterado irá notificar automaticamente todos os observadores, em particular, o sistema de navegação. De modo a preservar a imutabilidade do estado da interface, todas as alterações são efetuadas através do método *copy()*.

As operações disponibilizadas ao cliente estão implementadas por funções públicas, isto é, todas as ações que podem ser desencadeadas por um clique do cliente na interface gráfica, por exemplo, numa interação durante o jogo. Estes métodos iniciam *corrotinas* ao invés de interromperem a interface durante a execução das devidas operações, algumas delas potencialmente demoradas. Como já tinha sido referido anteriormente, nem todas as ações desencadeadas pelo cliente resultam numa chamada ao servidor, e limitam-se a atualizar o estado local da interface gráfica.

Finalmente, o *ViewModel* também disponibiliza uma função auxiliar de carregamento de recursos gráficos, nomeadamente, as imagens das peças ou personagens do jogo. Embora esta operação não altere o estado da aplicação, não deixam de provocar alterações na interface gráfica do utilizador, e também permite que os componentes da UI se foquem exclusivamente na apresentação de dados, reforçando novamente a importância dada ao *ViewModel* no que diz respeito às suas responsabilidades.

Obviamente, a gestão dos estados da interface gráfica e qualquer outra tarefa do *ViewModel* é partilhado entre todos os clientes, logo, este componente está presente no módulo '*common-Main*' de *composeApp*.

4.2.4 Ecrãs

De forma geral, os ecrãs seguem o padrão de receber estado como parâmetros, e comunicar eventos através de funções, que são eventualmente associadas a funções do *ViewModel*. Assim sendo, os ecrãs tornam-se mais simples e reutilizáveis, uma vez que não implementam nenhuma lógica interna, isto é, como toda a lógica de negócio está centralizada no *ViewModel*, os elementos da interface não têm qualquer controle sob o estado da aplicação, e focam-se apenas na apresentação dos elementos que lhes são delegados.

Há ecrãs puramente declarativos, que não possuem qualquer tipo de estado interno e apenas apresentam a informação que lhes é entregue, completamente ignorantes dos restantes componentes que os rodeiam. Outros, no entanto, necessitam de algum estado interno, estado este que é exclusivo à interface gráfica. Os estados mais acima na hierarquia, e que dependem diretamente do estado da aplicação, possuem responsabilidades maiores, e por vezes efetuam várias operações para delegar às outras componentes a informação que eles precisam.

Dado que os ecrãs apresentados são os mesmos, independentemente do cliente, todos eles fazem parte do módulo partilhado '*commonMain*' de *composeApp*.

4.3 BackEnd/Servidor

4.3.1 Base de dados

A base de dados do projeto foi implementada com o uso da biblioteca *Exposed*, para definir as tabelas que constituem a BD e para comunicar com a mesma.

Esta segue na sua maioria um modelo relacional, no entanto existem alguns objetos armazenados como *json*, ou seja são opacos à base de dados. Isto encontra-se apenas presente para os objetos Game, que têm uma complexidade bastante alta, e que necessitam de uma grande quantidade de informação persistida por cada um. Foi por este motivo, que implementámos a representação deste objeto na base de dados desta maneira: para facilitar o desenvolvimento. A implementação reduz o número de tabelas necessário, pois os objetos guardados em *json* foram apenas aqueles que tinham uma difícil representação e que, têm relação 1-1 com a tabela jogo, isto diminui a quantidade de *queries* e lógica necessária para obter e guardar um jogo da base de dados.

No entanto é necessário admitir que esta implementação é problemática e que a longo prazo os problemas são maiores que os benefícios, pois impossibilita de no futuro realizar pesquisa baseada em qualquer um dos objetos guardados como *Json* sem deserializar e serializar o objeto primeiro, ou alterar a definição da base de dados.

Por este motivo apesar da simplicidade oferecida por esta implementação e do tempo ganho para focar em outras partes do projeto, gastando menos tempo na base de dados em si tanto na implementação inicial como a implementar subseqüentes mudanças aos objetos representados dessa maneira, é um facto que a implementação deveria ser melhorada.

A biblioteca *Exposed* através da sua *DSL* trata de toda a interação com a base de dados sem necessitarmos de escrever quaisquer *queries SQL*, trata também da serialização e deserialização dos objetos *json* armazenados.

Foram realizados testes à base de dados, que foram principalmente utilizados para desenvolver a implementação inicial da mesma, no entanto alguns destes encontram-se desatualizados, pois não voltou a surgir a necessidade de os utilizar. É verdade que devíamos ter os testes atualizados, no entanto estes foram úteis na mesma. Futuras versões da base de dados deveriam ter ambos os pontos corrigidos, a base de dados deverá ser 100% relacional, e os testes deverão estar atualizados.

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do utilizador
name	VARCHAR(255)	NOT NULL	Nome de exibição do utilizador
email	VARCHAR(255)	UNIQUE, NOT NULL, Índice	Endereço de email do utilizador
password	VARCHAR(255)	NOT NULL	Palavra-passe encriptada
creation_date	LONG	NOT NULL	Data de criação da conta

Tabela 4.1: Tabela Users

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único da sessão de autenticação
token	VARCHAR(255)	UNIQUE, Índice	Token de autenticação
user_id	INT	Chave Estrangeira (Users.id), UNIQUE, Índice	Referência ao utilizador
game_id	INT	Nullable	Referência ao jogo ativo
token_refresh_time	LONG	NOT NULL	Data da última renovação do token
token_expiration	LONG	NOT NULL	Data de expiração do token

Tabela 4.2: Tabela AuthUsers

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único da mesa
name	VARCHAR(255)	UNIQUE, NOT NULL, Índice	Nome da mesa/sala de jogo
owner	INT	Chave Estrangeira (Users.id), NOT NULL	Utilizador que criou a mesa
capacity	INT	NOT NULL	Número máximo de jogadores

Tabela 4.3: Tabela Tables

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do participante
user_id	INT	Chave Estrangeira (Users.id), NOT NULL	Referência ao utilizador
lobby_id	INT	Chave Estrangeira (Tables.id), NOT NULL	Referência à mesa de jogo
role	VARCHAR(10)	NOT NULL	(PLAYER/SPECTATOR)

Tabela 4.4: Tabela Participants

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do jogo
turn	JSON	NOT NULL	Estado do turno atual (TurnDTO)
item_deck	JSON	NOT NULL	Array de itens no baralho
tile_deck	JSON	NOT NULL	Array de peças no baralho
battle	JSON	Nullable	Estado da batalha atual (BattleDTO)

Tabela 4.5: Tabela Games

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do jogador
game_id	INT	Chave Estrangeira (Games.id), CASCADE	Referência ao jogo
user_id	INT	Chave Estrangeira (Users.id), CASCADE	Referência ao utilizador
player_hands	JSON	NOT NULL	Array de cartas na mão do jogador
current_character	VARCHAR(255)	Nullable	Nome da personagem atualmente selecionada

Tabela 4.6: Tabela GamePlayers

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do espectador
game_id	INT	Chave Estrangeira (Games.id), CASCADE	Referência ao jogo
user_id	INT	Chave Estrangeira (Users.id), CASCADE	Referência ao utilizador

Tabela 4.7: Tabela GameSpectators

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único da peça do tabuleiro
game_id	INT	Chave Estrangeira (Games.id), CASCADE	Referência ao jogo
x	INT	NOT NULL	Coordenada X no tabuleiro
y	INT	NOT NULL	Coordenada Y no tabuleiro
board_tiles	JSON	NOT NULL	Dados da peça (TileDTO)
character_name	VARCHAR(255)	Nullable	Nome da personagem nesta peça

Tabela 4.8: Tabela BoardTiles

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do item
board_tile_id	INT	Chave Estrangeira (Board-Tiles.id), CASCADE	Referência à peça do tabuleiro
name	VARCHAR(255)	NOT NULL	Nome do item

Tabela 4.9: Tabela BoardCharacterItems

Atributo	Tipo	Restrições	Descrição
id	INT	Chave Primária, Auto-incremento	Identificador único do modificador
character_name	INT	Chave Estrangeira (Board-Tiles.id), CASCADE	Referência à peça do tabuleiro (personagem)
stats	VARCHAR(50)	NOT NULL	Estatísticas afetadas pelo modificador
duration	INT	NOT NULL	Duração do modificador (turnos)

Tabela 4.10: Tabela BoardCharacterModifiers

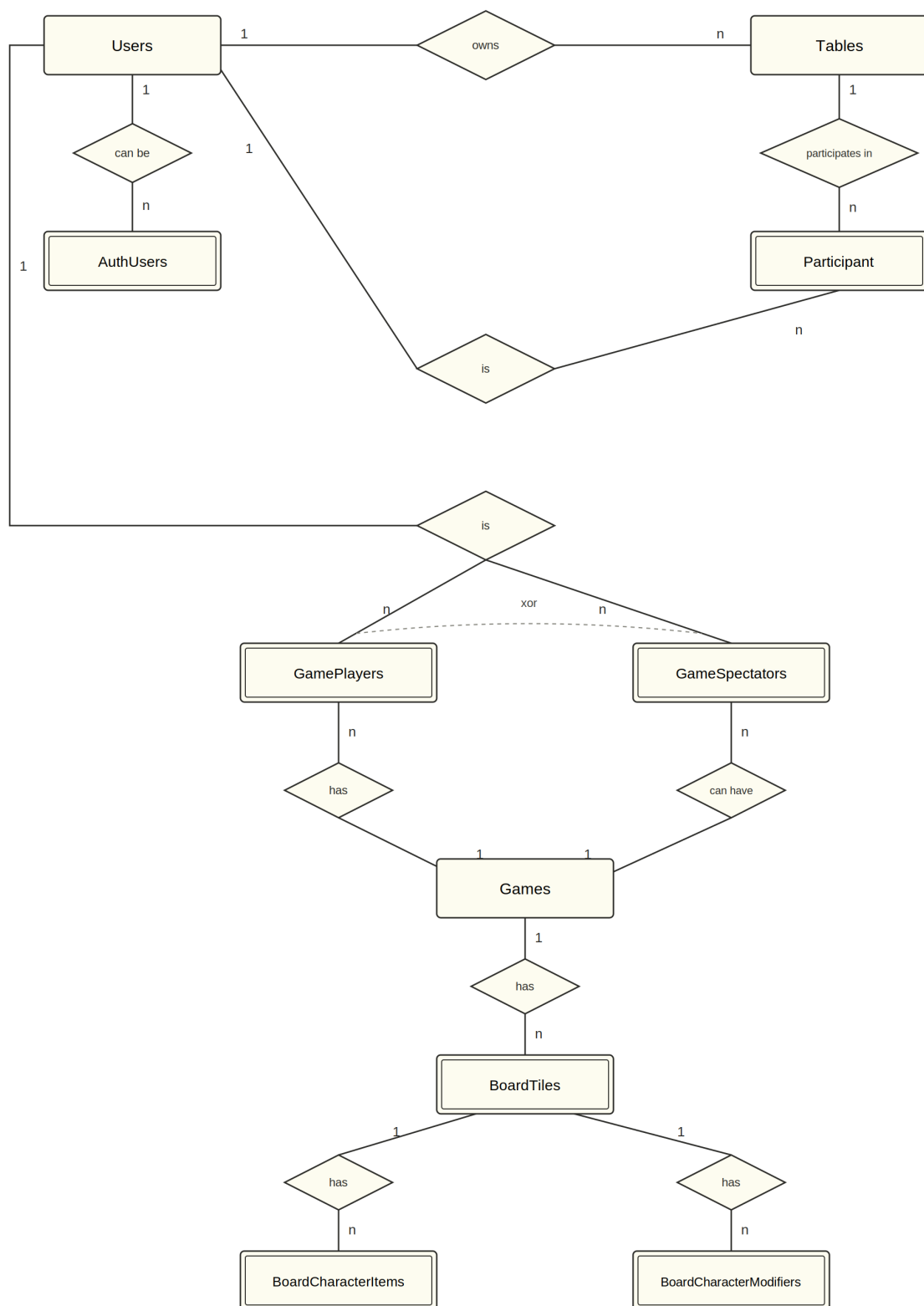


Figura 4.9: Diagrama EAbd

4.3.2 Autenticação

A nossa autenticação segue um esquema desenvolvido por nós para se aplicar da melhor forma possível às necessidades do nosso projeto.

No contexto do nosso projeto, a autenticação e segurança serve principalmente o papel de garantir que nenhum utilizador é capaz de interagir com um jogo de forma maliciosa. Tirando isso, a autenticação é apenas utilizada para identificar quais utilizadores é que estão *logged-in*, impedindo tentativas de *login* enquanto uma sessão atual permanece ativa. Removemos também de forma automática e periódica os utilizadores cuja sessão terminou, no entanto, não realizamos isso se o utilizador se encontrar dentro de um jogo que ainda não terminou.

Explicando a autenticação em si, fora do jogo apenas é verificado se o utilizador está autenticado ou não, com o parâmetro *auth* do mesmo. Este encontra-se em *null* sempre que o utilizador perder o *login* por qualquer razão.

Quando um jogador entra no jogo, a autenticação passa a ser validada com uma combinação do *token* de autenticação e de um *id* de jogo e tempo de fim de sessão. A *id* do jogo tem de ser equivalente aquela do jogo em que o jogador entrou. É também utilizado um *Instant* para identificar quando o *token* deve ser substituído por um novo garantindo que *token* válido é substituído de forma periódica. Este *instant* intitulado de *refreshTime* também foi considerado ser utilizado, para identificar inatividade. Alterando-o com todas as ações de jogo, se tiver um valor muito antigo, isso implica um cliente inativo. É possível então, através desse valor calcular então exatamente quanto tempo passou desde a última ação de jogo que um qualquer cliente realizou e utilizar esse tempo para definir um máximo possível aceitável para este se encontrar inativo em jogo, e fora de jogo, sendo estes estados facilmente identificados com o parâmetro *gameId* que é *null* quando o jogador está fora de um jogo. Isto poderia ser realizado tanto com o dito *refresh* deste valor, mas mais corretamente para impedir o *token* de ser substituído a cada ação, seria melhor realizado utilizando um terceiro instante para o propósito de identificar inatividade.

Apesar de ser menos ortodoxa do que uma implementação de autenticação padrão, utilizando por exemplo bibliotecas como o *JWT* ou mesmo aplicando uma mais autenticação simples com apenas *tokens* e fim de sessão, que já implementamos em vários projetos durante o curso, para o nosso projeto acreditamos que funciona tão bem ou melhor do que qualquer outro, pois tem os benefícios de garantir a integridade do jogo, enquanto se mantém simples na sua implementação, e permite obter na mesma bastante informação sobre a sessão de um qualquer cliente dependendo das necessidades.

Escolhemos implementar um sistema de autenticação exatamente para que esta tenha benefícios explicitamente para o nosso projeto, sendo os benefícios que ganhamos com esta, na

nossa opinião, maiores do que quaisquer benefícios que um sistema mais “padrão” pudesse oferecer. Isto, pois, necessidade de autenticação vai sempre ser diferente de caso em caso, e apesar de no geral estas necessidades serem semelhantes para a grande maioria dos casos, não são semelhantes para o nosso. Não existe grande necessidade de garantir integridade fora de um jogo, e desde que o nosso sistema dê fácil acesso à informação da sessão dos clientes, especialmente sobre a sua atividade para remover aqueles que se encontram inativos, é um bom sistema. Apesar de isto poder ser considerado um problema de segurança, não é um que consideramos grave, nem que valia a pena ser resolvido no contexto do nosso projeto.

Como foi mencionado pensamos em alternativas deste sistema, e ter um sistema pouco complicado, ajuda a surgindo a necessidade, como por exemplo de identificar utilizadores inativos, alterar o sistema para satisfazer dita necessidade.

4.3.3 Comunicação cliente-servidor

A comunicação entre cliente e servidor é realizada através das rotas expostas pelo servidor que definem a API da aplicação. Os clientes utilizam a mesma para comunicar com o servidor e realizar as operações desejadas. São utilizados *data transfer objects* (DTOS), representações *serializaveis* de objetos do domínio necessários na comunicação ou conjunto de informação necessária para cada pedido. Estes ajudam a generalizar a informação necessária, e minimizam os dados enviados, pois cada objeto apenas contém o estritamente necessário para representar um qualquer objeto do domínio, sendo o resto da informação calculada. Do lado do cliente foi implementada a *clientAPI*, o modulo responsável por determinar quais rotas são definidas para quais ações. Desta forma, para mudar qualquer rota, é apenas necessário alterar qual é chamada em *clientAPI*, e todas as funções que necessitam dessa ação irão utilizar dita rota. Isto simplifica alterações quer na API da aplicação, quer na definição de cada ação, pois a lógica de acesso ao servidor se encontra centralizada.

Rotas da nossa API:

- **Game API (/game)**
 - POST /game/create
 - POST /game/place
 - POST /game/rotate
 - POST /game/end-turn
 - POST /game/leave
 - POST /game/unequip
 - POST /game/applyGameUpdater
- **Table API (/tables)**
 - GET /tables
 - POST /tables/create
 - POST /tables/join
 - POST /tables/leave
 - POST /tables/change-role
- **User/Auth API (/auth/users)**
 - POST /users/create
 - POST /users/login
 - POST /users/logout
- **WebSocket API (/ws)**
 - websocket /ws
- **Static Resources (/assets/drawable)**
 - staticResources /assets/drawable

4.3.4 Controle de Erros

O servidor implementa um mecanismo de manuseamento de erros para garantir uma gestão consistente das exceções lançadas durante a sua execução, separando a lógica de negócio da lógica de comunicação. Na camada de serviço, todas as operações são executadas recorrendo à função *runCatching*, que devolve um objeto *Result*. Se durante uma das muitas validações efetuadas dali em diante, há alguma que falhe, o método de serviço capta o erro lançado e retorna o erro, senão, retorna o objeto que é esperado.

Para tratar do tratamento de erros, foi criada uma hierarquia de exceções baseados em *Error*, da qual derivam classes especializadas como *UserError* ou *GameError*. As exceções têm todas uma mensagem detalhada que permite identificar onde foi lançada a exceção e qual a causa do erro. O tratamento de todas as exceções é centralizado no momento de construir uma resposta HTTP, realizado através de um *plugin StatusPages* do *ktor*. Esta abordagem permite a adição fácil de novos tipos de erro, e garante o tratamento adequado das mesmas.

4.3.5 Testes

A implementação de testes unitários ao projeto, foram um dos pontos do projeto que podia ter sido muito melhorado. Foram desenvolvidos testes, e certas partes do projeto têm boa cobertura, no entanto outras encontram-se ou sem testes, com testes desatualizados ou com testes que têm baixa cobertura. Nos locais que contém ou chegaram a conter boa cobertura os testes foram úteis, no entanto manter os testes atualizados foi algo que não recebeu a atenção necessária. Apesar de não termos visto a necessidade de colocar esforço na implementação de testes, é um facto que este ponto deveria ser melhorado. Os pontos do projeto em que testes foram mais utilizados e mais úteis foram:

- Para testar o funcionamento das funções construídas de forma polimórfica;
- No teste do repositório base de dados;
- Para algumas das funcionalidades do jogo;

Apesar de nestes pontos os testes foram úteis alguns, acabaram por se encontrar eventualmente desatualizados, pois depois da sua utilização inicial, não foi vista a necessidade de os atualizar. Dado isto em futuras atualizações do projeto, seria ideal corrigir este problema, aumentando a cobertura dos testes para cobrirem todos os pontos da aplicação necessários.

4.4 BackEnd/Shared

4.4.1 Domínio

Dada a complexidade lógica do nosso projeto, a implementação do domínio foi um dos maiores desafios, dado o nosso objetivo de realizar uma aplicação facilmente expansível. Tentámos utilizar interfaces e tipos genéricos sempre que possível, e centralizar a maior quantidade de lógica em poucas classes, para que as outras possam ser mais simples, legíveis, e intuitivas.

Autenticação e Gestão de Mesas de Jogo

O utilizador precisa de estar autenticado para aceder à maior parte da aplicação, e para isso, ele tem de criar um *User*, e uma vez autenticado, ele tem a capacidade de criar ou juntar-se a uma *Table*. Nesta secção, estas duas classes são as que centralizam a maior parte da informação, como é fácil de perceber pela *Figura 4.10*.

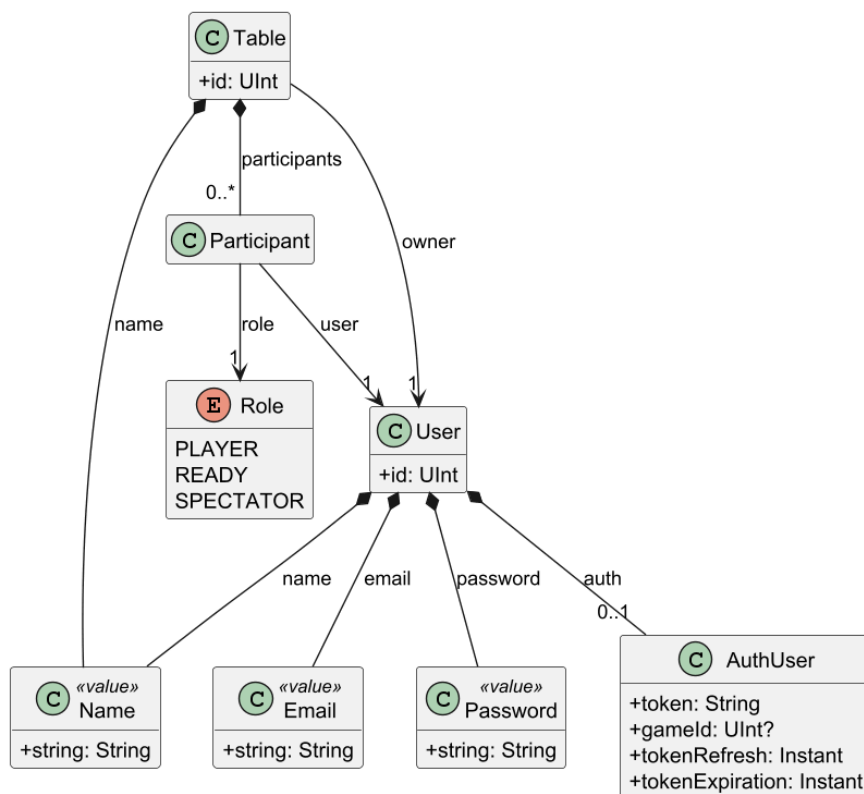


Figura 4.10: Diagrama PUMML das classes Table e User

Jogo

A autenticação já foi explicada anteriormente, e as Mesas de jogo não têm nada de especial a nível de elemento de Domínio. O mesmo não pode ser dito para os restantes componentes

do projeto, no entanto. Antes de falarmos da classe *Game*, a que encapsula a lógica total do jogo, vamos verificar os seus componentes um a um, começando pelo *Board*, na *Figura 4.11*.

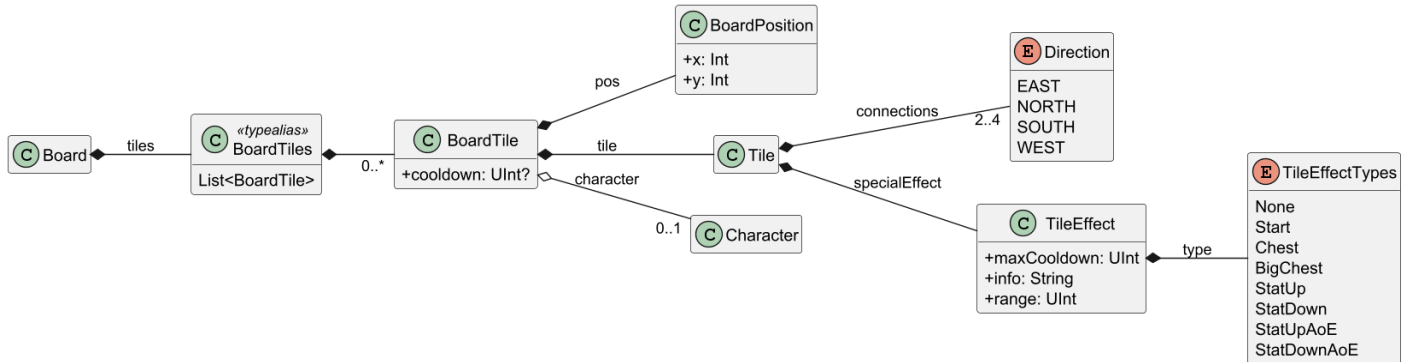


Figura 4.11: Diagrama PUMML da classe Board

O tabuleiro de jogo é composto pela lista de peças já colocadas. Cada peça tem informação acerca da sua posição relativa à origem do referencial, a peça *Start* que começa no tabuleiro em todos os jogos. Uma peça é definida pelo conjunto de direções por onde se pode conectar com outras peças, e pode ou não ter um efeito especial - o efeito que ativa quando um personagem passa por ela, personagem esse que está descrito na *Figura 4.12*.

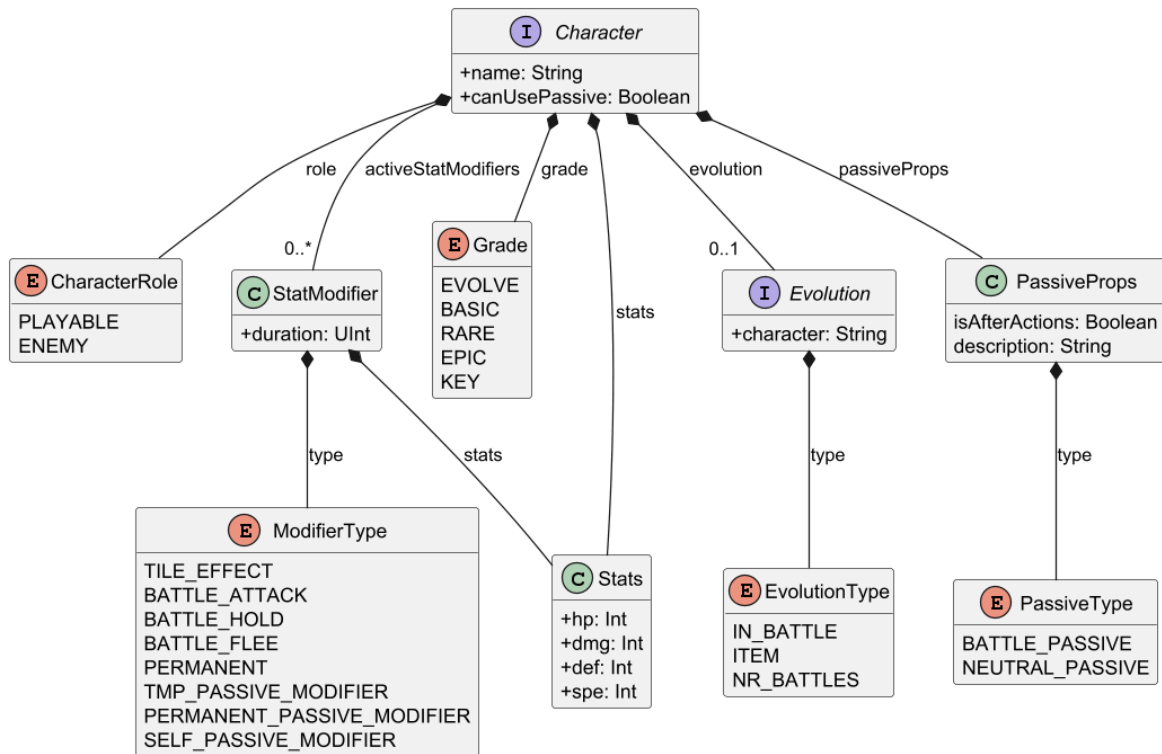
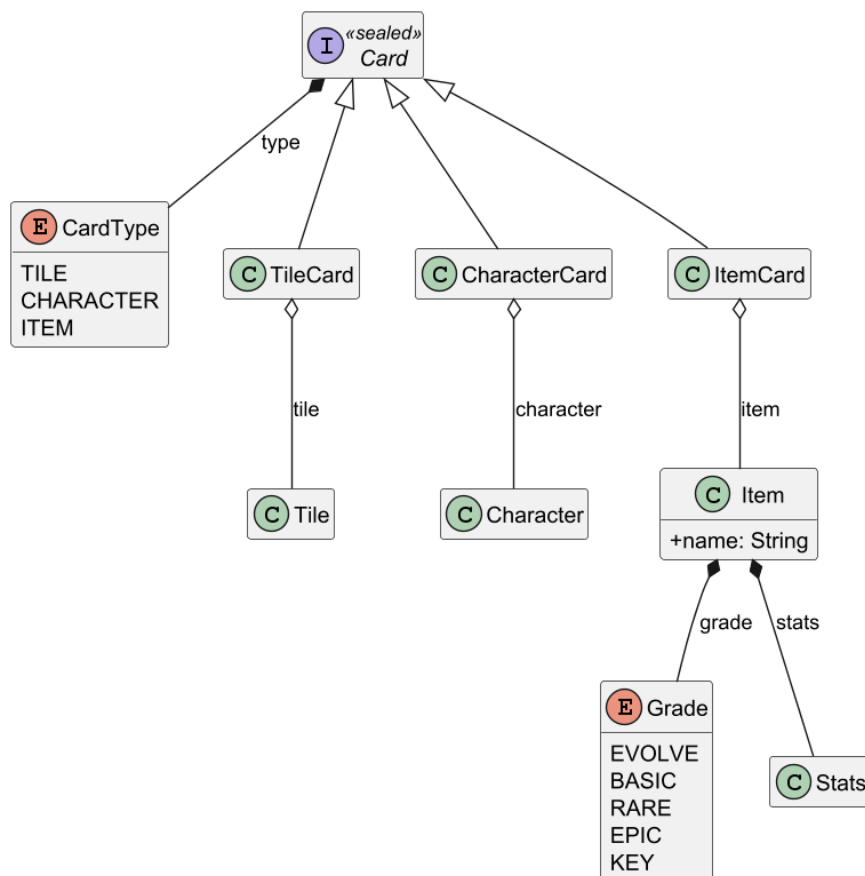


Figura 4.12: Diagrama PUMML da interface Character

Como não podia deixar de ser num jogo de cartas, também há uma interface de domínio que permite a adição de novos tipos de carta ao jogo. Atualmente, e como se pode verificar na *Figura 4.13*, há apenas três tipos de carta.



33

Antes de verificar a estrutura completa do jogo, resta-nos falar acerca de uma das mecânicas principais - as Batalhas, cujo diagrama *PUML* está presente na *Figura 4.14* abaixo.

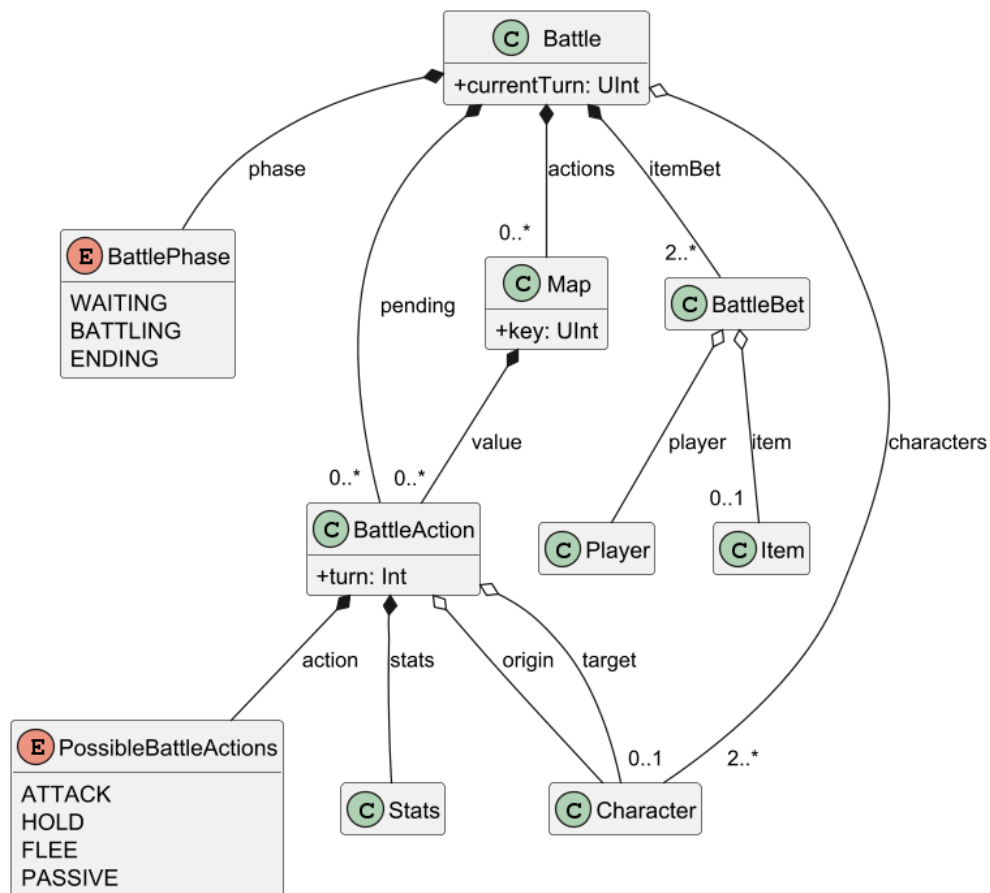


Figura 4.14: Diagrama PUML da classe *Battle*

As batalhas envolvem, logicamente, um conjunto de personagens e ações realizadas por esses personagens. Todos os jogadores com personagens a participar em batalha têm de colocar um dos seus itens (se tiverem) como aposta, o que significa que quanto mais personagens participarem, melhor é a recompensa por sair vencedor.

Finalmente, apresentamos na *Figura 4.15* o diagrama da classe *Game*.

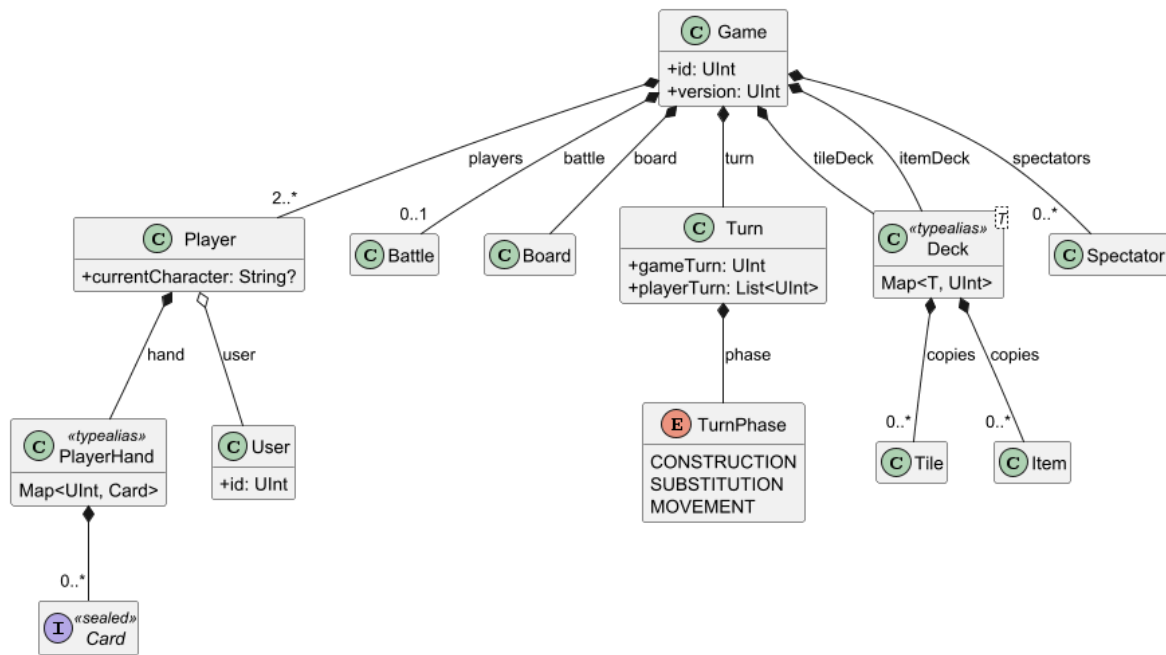


Figura 4.15: Diagrama PUML da classe Game

4.4.2 Expansibilidade

Um grande foco na nossa implementação foi a expansibilidade do código, foi claro desde o início que implementar tudo o que tínhamos planejado seria difícil, então decidimos colocar o foco em garantir que melhorias e adições futuras seriam o mais simples possível, para esse efeito foram tomadas várias decisões de *design* que, apesar de terem complicado a implementação, tinham o propósito de tornar o código facilmente expansível. Polimorfia em conjunção com reflexão foram a principal maneira de cumprir este objetivo, para além disso, com estas duas ferramentas, e desenhando o código de maneira a que quaisquer adições de funções que causam modificações simples ao jogo, podem ser adicionadas sem a necessidade de criar novas rotas.

Para além disso como já foi mencionado, o desenho geral do domínio está também construído para este propósito, mantendo as definições dos objetos de forma o mais geral possível tentando reduzir o número de objetos principais, e reduzindo informação redundante em cada um que complicaria expandir os mesmos.

Polimorfia

Polimorfia foi extremamente útil para o propósito de fazer o código expansível, utilizamos técnicas polimórficas para:

A abstração *Entity*: Utilizada para representar diversos objetos do jogo sobre o mesmo tipo. É a base dos principais usos de polimorfia no projeto, Foi essencial para permitir que as Interfaces

polimórficas implementadas possam afetar o máximo de objetos possível, expandindo os usos possíveis das mesmas. Esta abstração engloba os objetos:

- *Player*;
- *BoardTile*;
- *Board*;
- *Character*: Inclui todos os tipos de personagem atual (*PlayableCharacter*) e futuros como personagens inimigos controlados automaticamente cada um com suas *Stats* e passivas;
- *Card*: Inclui todos os tipos de carta;
- *BattleAction*;
- *Battle*;
- *Game*;

Definir funções que causam mudanças simples ao jogo de forma geral através da interface *UpdaterE* e da abstração *Entity*, esta interface é uma interface funcional, permitindo que novas implementações sejam ainda o mais simples possível;

Na definição das cartas de jogo sendo possível adicionar novos tipos de carta;

Nas passivas dos personagens, sendo possível implementar passivas que afetam qualquer objeto que pode ser representado pela abstração *Entity* tal como *UpdaterE* esta interface é uma interface funcional, pela mesma razão que *UpdaterE* é uma interface funcional, simplifica a implementação e facilita a leitura. É graças à existência de *Entity*, e ao desenho do domínio que possibilita que futuros personagens implementados tenham passivas que vão desde adicionar simples *Modifiers* que afetam as suas *Stats*, até afetar itens de outros jogadores, ou mesmo gerar *TileEffects*. Futuras implementações poderiam até ter habilidades que são ativadas pelos próprios jogadores, sendo apenas necessário criar uma função para ativar as mesmas, algo que seria relativamente simples graças à existência de *UpdaterE*, comparando à dificuldade de implementar a mesma funcionalidade sem a existência de *UpdaterE* e *Entity*. Infelizmente não tivemos tempo para implementar exemplos de todas estas passivas mais complexas, apesar do foco em simplificar a implementação das mesmas. No entanto foi ter noção desta dificuldade que nos levou a fazer implementar futuras passivas com diversidade extremamente alta, o mais simples possível, pois um qualquer programador, seria capaz de desenvolver as mesmas sem necessariamente ser familiar com todo o projeto.

Reflexão

Reflexão foi utilizada para que todas as funções classificadas como *UpdaterE* possam ser detetadas em tempo de compilação, e associadas ao seu nome, para que seja possível chamar ditas funções apenas com o seu nome reduzindo o numero de rotas necessárias. Graças a reflexão e à abstração *Entity*, com apenas uma única rota é possível definir quaisquer funções de jogo desde que tenham dois ou menos tipos que possam ser representados por *Entity* e que devolvam um jogo. Neste momento, existem 12 funções que de outra forma necessitariam de rotas para cada uma ou de associar cada uma ao seu nome manualmente. Considerando que no total, acabamos com cerca de 15 rotas incluindo a que chama todos os *UpdaterE*, o nosso uso de reflexão e polimorfia, reduziu em quase metade o numero de rotas necessárias, mesmo considerando que para chamar uma rota que utiliza um *UpdaterE*, é necessário passar em parâmetro o nome da função a ser chamada.

Capítulo 5

Conclusões

5.1 Dificuldades no projeto

As principais dificuldades que foram surgindo ao longo da implementação do projeto, advém principalmente da pouca ou nenhuma familiaridade que tínhamos com os componentes com que estávamos a trabalhar. A nossa experiência com *Ktor* tinha sido bastante limitada, e foi a primeira vez que usámos tanto a vertente multiplatforma do *Kotlin* como a biblioteca *Exposed*.

Um bom exemplo que nos levou a compreender melhor o objetivo da arquitetura multiplatforma foi quando começámos a trabalhar no cliente. Até então, o Domínio estava dentro do servidor junto dos outros módulos, como estávamos habituados a fazer, no entanto, rapidamente nos apercebemos que iríamos repetir imensa lógica no cliente que podia ser, perfeitamente aproveitada do domínio, ou teríamos de fazer operações muito mais complexas no servidor para obter uma resposta adequada à necessidade de apresentação da UI no cliente. Uma vez encontrada esta barreira, fomos capazes de entender intuitivamente a necessidade e a utilidade deste tipo de *framework*.

Da mesma maneira, a utilização da biblioteca *Exposed* pareceu-nos uma boa ideia porque removia a necessidade de escrever *queries* manualmente, o que para nós era uma mais valia. Quando realmente começámos a utiliza-la, vimos que a biblioteca era mais do que um tradutor de *Kotlin* para SQL, e que podíamos tirar proveito das suas outras funcionalidades para agilizar o processo de desenvolvimento e aumentar a segurança do código a possíveis erros, por exemplo, utilizando *Data Access Objects*.

Construir o código com foco em expansibilidade, também foi um desafio, pois é algo que até a data não tinha sido um foco de outro projeto que realizamos. Tivemos de juntar várias das técnicas que aprendemos ao longo do curso como polimorfia reflexão e programação funcional, e pensar como as aplicar para cumprir os nossos objetivos, desenvolvendo um tipo de estrutura

que não tínhamos construído até a este projeto.

Tendo sido claro que implementar todos os objetivos iniciais do jogo em si, decidimos focar-nos em fazer preparar o código para o futuro, para quer tivéssemos tempo ou não para implementar ditos objetivos, esta implementação ser mais simples. Foi uma escolha tomada bastante no início do projeto, e grande parte da razão de termos posto tanto foco do desenvolvimento em construir a aplicação à volta de expansibilidade. Escolher em quais partes do projeto queríamos por o foco, foi por vezes complicado, pois implicava escolher quais partes da nossa ideia inicial eram mais importantes para a identidade do projeto, algo que em certos casos era óbvio mas noutros não.

Ter que escolher quais das ideias iniciais seriam sacrificadas em função de melhorar a qualidade geral do projeto, foi, desta forma, uma das grandes dificuldades ao longo de todo o desenvolvimento, pois várias parte do projeto que implementámos, foram implementadas, não por necessidade, mas sim por escolha, escolha que nem sempre foi fácil ou óbvia. É então um facto que devido a esta dificuldade com que nos deparamos, é debatível que algumas das funcionalidades que escolhemos implementar, não foram necessariamente a melhor escolha, havendo mais do que uma opção correta. E que, tendo tido mais tempo ou tendo nós sido capazes de acelerar o desenvolvimento, talvez não tivéssemos a necessidade destas escolhas de todo. No entanto, tomamos as decisões que tomamos de forma deliberada, e estamos preparados para explicar cada uma delas, e acreditamos que o que o trabalho que entregamos, reflete aquilo que fomos capazes de construir dado o tempo que tivemos para a implementação do projeto, e a necessidade de trabalhar nas nossas outras responsabilidades.

5.2 Objetivos não implementados

5.2.1 Lógica de Jogo

Infelizmente, constrangidos pela quantidade limitada de tempo, fomos incapazes de implementar tudo aquilo que estava planeado quando a ideia foi proposta. A ideia principal de jogo permaneceu inalterada, mas houve muito conteúdo que teve de ser alterado ou colocado totalmente de parte.

O jogo acabou por ser uma experiência competitiva onde há muito pouco espaço para colaboração entre os jogadores, mas a ideia original era ter missões que introduziam condições de vitória completamente diferentes e faziam surgir inimigos controlados automaticamente pela aplicação que obrigavam os jogadores a trabalhar em conjunto temporariamente.

Além dos inimigos, houve alguns personagens que tinham sido planeados no início que acabaram por não ser introduzidos na versão final do jogo, dando menos alternativas e estilos de jogo para os jogadores tirarem proveito. Assim como os personagens, itens também eram

para ter um sistema de progressão, onde ficavam naturalmente mais fortes, e até o tabuleiro teve de sofrer alterações devido a efeitos que nunca foram implementados.

5.2.2 Aspetos técnicos

Implementação de autenticação *Oauth*: Não chegámos a integrar autenticação *Oauth* na versão final do projeto, isto deveu-se acharmos que esta, tinha pouca relevância no contexto do mesmo, e que poderia limitar a forma como implementávamos a nossa autenticação. Por este motivo, apesar de termos realizado pesquisa de como o fazer, tendo iniciado a implementação. e de já tendo construído um projeto com autenticação *Oauth* na cadeira de Segurança Informática, escolhemos não incluir *Oauth* como opção de autenticação. No entanto dado que é uma tecnologia útil em qualquer projeto que utilize autenticação, e que este era um dos objetivos iniciais pretendidos para o projeto, temos de reconhecer que a falta desta opção de autenticação, é um ponto negativo, mesmo que não consideremos autenticação um dos pontos principais do projeto.

5.3 Melhorias a fazer

Existem vários pontos no projeto que podem ser melhorados, alguns dos quais já foram mencionados ao longo do resto do relatório, listando todos esses mais aquelas que não foram mencionadas;

Identificação de jogadores inativos:

Melhoria de testes:

Este ponto já foi falado na secção de testes, os nossos testes ao longo de toda a aplicação encontram-se relativamente em falta. Seria necessário em futuras melhorias da aplicação, adicionar um grande numero de testes unitários, assim como atualizar alguns dos existentes, para garantir que existe cobertura suficiente para testar a aplicação de forma simples. Especialmente do lado da aplicação cliente existem muito poucos testes unitários, algo que seria essencial corrigir no futuro

Melhoria do UI/GUI:

O aspeto da UI da nossa aplicação foi algo que recebeu muito pouca atenção ao longo de todo o desenvolvimento da mesma. Apesar dos elementos nos ecrãs terem sido desenhados para ser reutilizados, os componentes menores como botões, caixas de texto, e outros componentes visuais individuais não têm uma definição específica na aplicação, são os objetos padrão oferecidos pelo *Jetpack Compose*.

Assim sendo, o processo de embelezar a interface gráfica da aplicação quando chegámos à fase final do desenvolvimento, revelou-se um processo demasiado demorado para valer o esforço, afinal, o objetivo não era entregar gráficos de última geração ou sequer animações. Fica como lição, no entanto, que se o aspeto visual for importante para o resultado final de um projeto como este, é importante criar interfaces customizadas de todos os objetos para serem facilmente manipuláveis, colocando todos os detalhes no construtor da classe.

Base de dados:

Como foi mencionado, a implementação da nossa base de dados, contém um grande problema: Não é 100% relacional. Como foi também mencionado, isto foi feito desta forma, para acelerar desenvolvimento, no entanto claramente não é a melhor implementação pois contém diversos problemas que podem surgir no futuro devido á mesma, e num projeto construído em grande parte à volta de futuras melhorias, isto é um problema relativamente sério.

É necessário substituir todos os objetos guardados na base de dados com representação *Json*, pois apesar destes ter alguns benefícios, tem o grande problema de serem opacos à base de dados, impossibilitando *queries* baseados nos mesmos, e a implementação de regras de negócio da base de dados para ditos objetos.

Esta implementação de objetos representados por *Json* deve ser substituída por uma representação relacional independentemente da necessidade de mais tabelas que isso implica, para que siga as regras de uma base de dados relacional.

Comunicação WebSocket:

A comunicação *WebSocket* implementada foi a primeira solução que encontrámos, mas não foi a melhor. Atualmente, quando é efetuada uma mudança que tem de ser transmitida aos clientes, o objeto que sofreu as alterações é enviado na sua totalidade, mesmo quando se trata de uma alteração mínima. Dando um exemplo, quando os participantes de uma mesa esperam para o jogo começar, os jogadores têm de sinalizar a sua intenção de começar o jogo, que é feito via clique na interface gráfica que, posteriormente, irá criar um pedido ao servidor para efetuar a mudança de estado do participante.

Após a mudança ser efetuada, o servidor envia o novo objeto da mesa para todos os participantes, quando podia enviar apenas um sinal a identificar a ação realizada. Apesar deste tipo de comunicação não ser viável para clientes que não sejam os da nossa aplicação, é possível fazer essa distinção e enviar o objeto completo aos clientes que necessitam de toda a informação, e apenas o sinal àqueles que são capazes de calcular o próximo estado com base no sinal recebido.

Não sendo algo difícil de implementar, por não ser estritamente necessário para a conclusão do projeto, foi algo que nunca chegou a ser implementado.

5.4 O que aprendemos

Apesar do que ficou por fazer e das melhorias que pretendíamos realizar, mas que nunca foram de facto efetuadas, o projeto realizado excedeu as nossas expectativas no sentido em que pensávamos que íamos colocar em prática o conhecimento que tínhamos desenvolvido ao longo dos últimos anos, mas na verdade, aprendemos ainda mais coisas nas áreas que pensávamos conhecer.

Aprendemos os desafios de criar código focado em expansibilidade, e como utilizar as ferramentas para cumprir esse propósito. Ao longo do projeto, vimos-nos forçados a ter em consideração a forma como a nossa implementação iria afetar trabalho futuro, fazendo-nos pensar e reavaliar todas as situações antes de começar realmente a implementar o que quer que fosse.

Apesar de não termos conseguido implementar todas as funcionalidades que tínhamos planeado, também aprendemos que o tempo necessário para desenvolver um projeto é muito maior do que se está à espera, e que o que pensámos que seria simples, se tornou um obstáculo com a introdução de novos conceitos e ideias, e que por vezes é necessário adaptar a estratégia e o plano para se adequar a estes novos obstáculos.

De qualquer modo, estamos gratos que de uma forma ou outra, com mais ou menos imprevistos, conseguimos obter um produto para todos os efeitos completo.

Referências

- [1] Mattel. Uno! official website, 2026. Accessed July 9, 2026.
- [2] Wizards of the Coast. Magic: The gathering official website, 2026. Accessed July 9, 2026.
- [3] Edmund McMillan. The binding of isaac official website, 2026. Accessed July 9, 2026.
- [4] International Chess Federation (FIDE). Fide official website, 2026. Accessed July 9, 2026.
- [5] Larian Studios. Baldur's gate 3 official website, 2026. Accessed July 9, 2026.
- [6] JetBrains. Kotlin multiplatform, 2026. Accessed July 9, 2026.
- [7] JetBrains. JetBrains official website, 2026. Accessed July 9, 2026.
- [8] JetBrains. Ktor official website, 2026. Accessed July 9, 2026.
- [9] http4k Team. http4k, 2026. Accessed July 9, 2026.
- [10] JetBrains. Exposed, 2026. Accessed July 9, 2026.
- [11] Jdbi. Jdbi 3 developer guide, 2026. Accessed July 9, 2026.
- [12] Google, JetBrains. Jetpack compose, 2026. Accessed July 9, 2026.